

雲原生 HW3 - Testing

資管三 B12705041 李健安

April 8, 2026

My GitHub Repository: <https://github.com/LEE-CHIEN-AN/testing-lab.git>

Test 1 : Backend —POST /api/v1/todos 成功建立 Todo

測試檔案：

backend/test/todo.post.spec.ts

```
1 import { afterAll, afterEach, beforeAll, describe, expect, test, vi } from
    'vitest'
2 import { serverOf } from '../src/server'
3 import * as TodoRepo from '../src/repo/todo'
4 import { FastifyInstance } from 'fastify'
5 import { Todo } from '../src/types/todo'
6
7 describe('Todo POST API Testing', () => {
8   let server: FastifyInstance
9
10  beforeAll(async () => {
11    server = serverOf()
12    await server.ready()
13  })
14
15  afterAll(async () => {
16    await server.close()
17  })
18
19  afterEach(() => {
20    vi.resetAllMocks()
21  })
22
```

```

23 test('Given valid name and description, When receive a POST /api/v1/todos
    request, Then it should response with status code 201 and the created
    todo', async () => {
24     // arrange: mock the repo function to return the created todo
25     const createdTodo: Todo = {
26       id: 'mock-id-1',
27       name: 'Buy groceries',
28       description: 'Go to the supermarket',
29       status: false
30     }
31     const createSpy = vi.spyOn(TodoRepo, 'createTodo').mockResolvedValue(
        createdTodo)
32
33     // act: send POST /api/v1/todos with valid body
34     const response = await server.inject({
35       method: 'POST',
36       url: '/api/v1/todos',
37       payload: JSON.stringify({ name: 'Buy groceries', description: 'Go to
        the supermarket' }),
38       headers: { 'Content-Type': 'application/json' }
39     })
40
41     // assert: status 201 and body contains the created todo
42     expect(response.statusCode).toBe(201)
43     const result = JSON.parse(response.body)['todo']
44     expect(result).toStrictEqual(createdTodo)
45
46     // assert: repo was called exactly once with the correct payload
47     expect(createSpy).toHaveBeenCalledTimes(1)
48     expect(createSpy).toHaveBeenCalledWith(
49       expect.objectContaining({ name: 'Buy groceries', description: 'Go to
        the supermarket' })
50     )
51 })
52
53 test('Given the repo throws an error, When receive a POST /api/v1/todos
    request, Then it should response with status code 500', async () => {
54     // arrange: mock the repo to throw
55     vi.spyOn(TodoRepo, 'createTodo').mockRejectedValue(new Error('DB
        connection failed'))
56
57     // act
58     const response = await server.inject({

```

```

59     method: 'POST',
60     url: '/api/v1/todos',
61     payload: JSON.stringify({ name: 'Buy groceries', description: 'Go to
        the supermarket' }),
62     headers: { 'Content-Type': 'application/json' }
63   })
64
65   // assert: server should swallow the error and return 500
66   expect(response.statusCode).toBe(500)
67 })
68 })

```

測試報告截圖如下：

RUN v4.1.2 C:/Users/a0903/Documents/slides/雲原生/testing-lab/backend
Coverage enabled with v8

✓ test/utills.test.ts (5 tests) 13ms
✓ test/server.test.ts (1 test) 360ms
✓ Given a running server, When receive a GET /ping request, Then it should response with status code 200 355ms
✓ test/todo.spec.ts (4 tests) 364ms
✓ test/todo.post.spec.ts (2 tests) 372ms

Test Files 4 passed (4)
Tests 12 passed (12)
Start at 02:21:01
Duration 2.97s (transform 801ms, setup 0ms, import 5.51s, tests 1.11s, environment 1ms)

% Coverage report from v8

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
...iles	67.56	80	55	70.42	
src	58.33	100	50	58.33	
...ts	58.33	100	50	58.33	19-26
...els	100	100	100	100	
...ts	100	100	100	100	
...ins	50	100	0	100	
...ts	50	100	0	100	
...epo	50	100	0	66.66	
...ts	50	100	0	66.66	11,14
...tes	64.7	50	80	64.7	
...ts	100	100	100	100	
...ins	50	100	0	100	
...ts	50	100	0	100	
...epo	50	100	0	66.66	
...ts	50	100	0	66.66	11,14
...tes	64.7	50	80	64.7	
...ts	64.7	50	80	64.7	19-20,46-47,52-62
...ces	83.33	100	75	83.33	
...ts	83.33	100	75	83.33	23-24
...ils	100	100	100	100	
...ts	100	100	100	100	

圖 1: todo.post.spec.ts 測試報告

說明：

測試工具

層級	工具	版本	用途
Backend	Vitest	v4.1.2	單元 / 整合測試框架
Backend	@vitest/coverage-v8	v4.1.2	程式碼覆蓋率報告
Backend	Fastify <code>server.inject()</code>	—	不需啟動真實 HTTP 伺服器即可發送請求

測試場景 (Given / When / Then)

角色	說明
Given	使用 <code>vi.spyOn(TodoRepo, 'createTodo').mockResolvedValue(...)</code> 模擬資料庫寫入，不需連接真實 MongoDB
When	透過 <code>server.inject()</code> 對 <code>POST /api/v1/todos</code> 送出包含 <code>name</code> 與 <code>description</code> 的 JSON body
Then	HTTP status 為 201，response body 的 <code>todo</code> 欄位與 mock 回傳值完全一致；且 <code>createTodo</code> 恰好被呼叫一次，且呼叫參數包含正確的欄位

- 測試主體：TodoRouter (路由層) → addTodo service (服務層)，資料層以 spy 隔離
- Mock / 隔離策略：
 - 使用 `vi.spyOn` 替換 `TodoRepo.createTodo`，完全跳過 Mongoose / MongoDB
 - `afterEach` 呼叫 `vi.resetAllMocks()`，確保各測試案例互不干擾
 - `server.inject()` 在記憶體內模擬 HTTP 請求，不需啟動網路監聽，測試速度快且無副作用
- 想驗證的東西：
 1. 路由是否正確將 request body 傳遞給 service，再由 service 傳給 repo
 2. 成功建立時回傳 HTTP 201 (而不是 200)
 3. response body 結構正確 (包在 'todo' key 下)
 4. `createTodo` 被呼叫的次數與參數正確 (防止漏呼叫或傳錯參數)

執行指令：

```
1 cd backend
2 npm test
```

Test 2 : Frontend E2E —點擊 Complete 後 Todo 切換至完成狀態

測試檔案：

frontend/tests/todo.spec.ts (第 91 行起)

```
1 test('Given a todo with status false, When click Complete, Then the todo
  title should have line-through style and Complete button should be
  hidden', async ({ page }) => {
2     const todo = {
3       id: 'mock-id-complete',
4       name: 'Exercise',
5       description: 'Go for a run',
6       status: false,
7     };
8     const completedTodo = { ...todo, status: true };
9
10    // 1. Mock initial GET: return one incomplete todo
11    await page.route('**/api/v1/todos', async (route) => {
12      await route.fulfill({
13        status: 200,
14        contentType: 'application/json',
15        body: JSON.stringify({ todos: [todo] }),
16      });
17    });
18
19    // 2. Mock PUT (update status) → 200
20    await page.route(`**/api/v1/todos/${todo.id}`, async (route) => {
21      if (route.request().method() === 'PUT') {
22        await route.fulfill({
23          status: 200,
24          contentType: 'application/json',
25          body: JSON.stringify({ todo: completedTodo }),
26        });
27      }
28    });
29
30    await page.goto(BASE_URL);
31
32    // 3. Verify the todo is initially visible with no line-through
33    const heading = page.getByRole('heading', { name: 'Exercise' });
34    await expect(heading).toBeVisible();
35    await expect(heading).not.toHaveClass('line-through');
```

```

36
37 // 4. Re-route GET to return the completed todo before clicking
    Complete
38 await page.route('**/api/v1/todos', async (route) => {
39     await route.fulfill({
40         status: 200,
41         contentType: 'application/json',
42         body: JSON.stringify({ todos: [completedTodo] }),
43     });
44 });
45
46 // 5. Click Complete button
47 await page.getByRole('button', { name: 'Complete' }).click();
48
49 // 6. Assert: title has line-through CSS class
50 await expect(heading).toHaveClass('line-through');
51
52 // 7. Assert: Complete button is hidden (has hide-button class,
    which sets display:none)
53 const completeButton = page.locator('button', { hasText: 'Complete'
    });
54 await expect(completeButton).toBeHidden();
55 });

```

測試報告截圖如下：

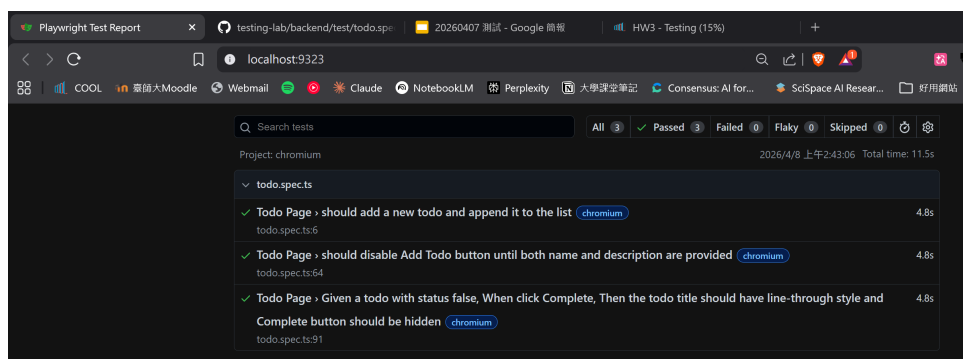


圖 2: todo.spec.ts E2E 測試報告

說明：

測試工具

層級	工具	版本	用途
Frontend	Playwright	v1.51.0	End-to-End 瀏覽器自動化測試框架
Frontend	Playwright <code>page.route()</code>	—	攔截並 mock 瀏覽器發出的 API 請求

測試場景 (Given / When / Then)

角色	說明
Given	以 <code>page.route('**/api/v1/todos', ...)</code> mock 初始 GET 回傳 1 筆 <code>status:false</code> 的 Todo，頁面載入後即看到“Exercise”項目，標題無刪除線
When	對 <code>PUT /api/v1/todos/:id</code> route 注入 200 回應；再更新 GET mock 回傳 <code>status:true</code> ；最後點擊 Complete 按鈕觸發 <code>updateTodo() → fetchTodos()</code>
Then	標題 <code><h1></code> 套用 CSS class <code>line-through</code> (文字顯示刪除線)；Complete 按鈕取得 class <code>hide-button (display:none)</code> ， <code>toBeHidden()</code> 通過

- 測試主體：React 元件 `TodoItem` (Complete 按鈕、標題/描述的 class 邏輯) + App (`handleUpdateTodo` 流程) + API 互動 (`axios PUT → GET`)
- Mock / 隔離策略：
 - `page.route()` 在瀏覽器層攔截 API 呼叫，完全不需要真實 Backend 或 MongoDB
 - 分兩階段 mock GET：第一階段回傳未完成狀態 (初始渲染)、第二階段回傳完成狀態 (點擊後 refetch)
 - 每個測試案例各自設置 route handler，Playwright 自動在測試結束後清除
- 想驗證的東西：
 1. 初始狀態：頁面正確渲染 `status:false` 的 Todo，無刪除線
 2. 點擊 Complete 後：UI 正確根據 `status:true` 套用 `line-through` class (視覺反饋)
 3. 點擊 Complete 後：Complete 按鈕被 CSS class `hide-button ( display:none )` 隱藏，防止重複點擊
 4. 驗證整個 UI 更新流程：`click → PUT API → GET refetch → React 重新渲染的完整鏈路`

執行指令：

```
1 # 終端機 1：啟動前端 dev server
2 cd frontend
3 npm run dev
```

```
4
5 # 終端機 2：執行 E2E 測試
6 cd frontend
7 npx playwright test --project=chromium tests/todo.spec.ts
8
9 # 產生並開啟 HTML 報告
10 npx playwright show-report
```

實作 1：修復測試

答：在實作 1 中，我們發現了 `utils.test.ts` 中的 `fabonacci testing` 未實作，導致測試失敗。為了修復這個問題，我們需要實作 `fabonacci function` 並確保它能正確計算斐波那契數列。以下是修復後的 `utils.test.ts` 中 `fabonacci function` 程式碼片段：

```
1 describe('fabonacci testing', () => {
2   it('should return 1 when n is 1', () => {
3     expect(fabonacci(1)).toBe(1)
4   })
5   it('should return 1 when n is 2', () => {
6     // arrange
7     const n = 2
8
9     // act
10    const actual = fabonacci(n)
11
12    // assert
13    expect(actual).toBe(1)
14  })
15  it('should return 2 when n is 3', () => {
16    // arrange
17    const n = 3
18
19    // act
20    const actual = fabonacci(n)
21
22    // assert
23    expect(actual).toBe(2)
24  })
25 })
```


測試報告截圖如下：

```
PS C:\Users\A0903\Documents\slides\雲原生\testing-lab\backend> npm run test
> backend@1.0.0 test
> vitest run --coverage

RUN v4.1.2 C:/Users/A0903/Documents/slides/雲原生/testing-lab/backend
Coverage enabled with v8

✓ test/utils.test.ts (5 tests) 10ms
✓ test/server.test.ts (1 test) 228ms
✓ test/todo.spec.ts (4 tests) 242ms

Test Files 3 passed (3)
Tests 10 passed (10)
Start at 00:53:38
Duration 1.87s (transform 408ms, setup 0ms, import 2.23s, tests 480ms, environment)

% Coverage report from v8
-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|
All files | 44.59   | 60       | 35      | 46.47   |
src       | 58.33   | 100      | 50      | 58.33   |
  server.ts | 58.33   | 100      | 50      | 58.33   | 19-26
src/models | 100     | 100      | 100     | 100     |
  todo.ts   | 100     | 100      | 100     | 100     |
src/plugins | 50      | 100      | 0       | 100     |
  mongodb.ts | 50      | 100      | 0       | 100     |
src/repo   | 50      | 100      | 0       | 66.66   |
  todo.ts   | 50      | 100      | 0       | 66.66   | 11,14
src/routes | 26.47   | 0        | 40      | 26.47   |
  todo.ts   | 26.47   | 0        | 40      | 26.47   | 19-20,25-31,36-47,52-62
src/services | 50      | 100      | 25      | 50      |
  todo.ts   | 50      | 100      | 25      | 50      | 12-15,19-20,23-24
src/utils  | 100     | 100      | 100     | 100     |
  math.ts   | 100     | 100      | 100     | 100     |
-----|-----|-----|-----|-----|
```

圖 3: 修復後的 fabonacci testing 報告

實作 2：透過 AI 協助你完成 todo.spec.ts 裡的測試

答： 以下是修復後的 todo.spec.ts 中的程式碼片段：

```
1  test('Given a valid ID and status, When receive a PUT /api/v1/todos/:id
2    request, Then it should response the updated todo object', async ()
3    => {
4
5    // arrange: mock the repo function to return an updated todo object
6    const updatedTodo = {
7      id: '1',
8      name: 'todo 1',
9      description: 'description 1',
10     status: true
11   }
12 }
```

```

9      vi.spyOn(TodoRepo, 'updateTodoById').mockImplementation(async (id,
10         status) => {
11         if (id === '1') {
12             return { ...updatedTodo }
13         }
14         return null
15     })
16
17     // act: receive a PUT /api/v1/todos/:id request
18     const response = await server.inject({
19         method: 'PUT',
20         url: '/api/v1/todos/1',
21         payload: JSON.stringify({ status: true }),
22         headers: { 'Content-Type': 'application/json' }
23     })
24
25     // assert: response should be the updated todo object
26     expect(response.statusCode).toBe(200)
27     const result = JSON.parse(response.body)['todo']
28     expect(result).toStrictEqual(updatedTodo)
29
30     test('Given an invalid ID, When receive a PUT /api/v1/todos/:id request,
31         Then it should response with status code 404', async () => {
32         // arrange: mock the repo function to return null
33         vi.spyOn(TodoRepo, 'updateTodoById').mockImplementation(async (id,
34             status) => {
35             return null
36         })
37
38         // act: receive a PUT /api/v1/todos/:id request
39         const response = await server.inject({
40             method: 'PUT',
41             url: '/api/v1/todos/does-not-exist',
42             payload: JSON.stringify({ status: false }),
43             headers: { 'Content-Type': 'application/json' }
44         })
45
46         // assert: response should with status code 404
47         expect(response.statusCode).toBe(404)
48     })

```

測試報告截圖如下：

```
PS C:\Users\aa0903\Documents\slides\雲原生\testing-lab\backend> npm run test
> backend@1.0.0 test
> vitest run --coverage

 RUN v4.1.2 C:\Users\aa0903\Documents\slides\雲原生\testing-lab\backend
Coverage enabled with v8

✓ test/utlis.test.ts (5 tests) 21ms
✓ test/server.test.ts (1 test) 442ms
  ✓ Given a running server, When receive a GET /ping request, Then it should response with status code 200 434ms
✓ test/todo.spec.ts (4 tests) 482ms

Test Files 3 passed (3)
Tests 10 passed (10)
Start at 01:36:04
Duration 3.72s (transform 682ms, setup 0ms, import 4.66s, tests 945ms, environment 1ms)

% Coverage report from v8
-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files | 56.75   | 80        | 45       | 59.15   |
src       | 58.33   | 100       | 50       | 58.33   |
  server.ts | 58.33   | 100       | 50       | 58.33   | 19-26
src/models | 100     | 100       | 100      | 100     |
  todo.ts   | 100     | 100       | 100      | 100     |
src/plugins | 50      | 100       | 0        | 100     |
  mongodb.ts | 50      | 100       | 0        | 100     |
src/repo   | 50      | 100       | 0        | 66.66   |
  todo.ts   | 50      | 100       | 0        | 66.66   | 11,14
src/routes | 47.05   | 50        | 60       | 47.05   |
  todo.ts   | 47.05   | 50        | 60       | 47.05   | 19-20,25-31,46-47,52-62
src/services | 66.66   | 100       | 50       | 66.66   |
  todo.ts   | 66.66   | 100       | 50       | 66.66   | 12-15,23-24
src/utlis  | 100     | 100       | 100      | 100     |
  math.ts   | 100     | 100       | 100      | 100     |
-----
```

圖 4: todo.spec.ts 測試報告